Sam Altman's Number

A finite number beyond Rayo, with syntactic and arithmetic Lean certificates

GPT-6 Astra

15 October 2026

Abstract

We define a finite natural number $S_{\mathcal{V}}$ and prove $2^{R_{\mathcal{V}}} < S_{\mathcal{V}}$, where $R_{\mathcal{V}}$ is Rayo's number under the intended interpretation of set theory used in his original definition. The construction uses truth on an explicitly finite collection of first-order formulas, a specified oracle-program language, the join of its finite diagonal-halting iterates, and a googol-bit strict output bound. An actual program of code $1851571349958786726 < 2^{61}$ searches a comparison channel and returns the least bound on the relevant named naturals. A second explicit program exponentiates this output and has code below 2^{2048}. Kernel-checked Lean proofs certify the finite syntax, logical uniqueness, program encoding, search behavior, and an unconditional arithmetic comparison for every input truth profile. The intended profile is supplied by finitely many Separation instances, with no identification of a Lean type with the universe of all sets. This specifies the semantic interpretation separately from the checked uniform certificate. No largest-number, record, or new general diagonalization method is claimed.

1 The benchmark and the result

Put $G = 10^{100}$. We use Rayo's final displayed winning entry from the 2007 Big Number Duel: descriptions have *fewer than* G symbols, one free variable, and exactly one satisfying object among all sets; that object must be a finite von Neumann ordinal. His introductory paraphrase has a different endpoint, so it is the final definition that fixes our benchmark [1]. Write $R_{\mathcal{V}}$ for the least natural number strictly greater than every natural named in this way, with the standard intended interpretation denoted by $\mathcal{V}$.

Theorem 1.1 (Main comparison)**.** *The construction in Definitions 3.1 and 5.2 determines a finite natural number* $S_{\mathcal{V}}$*, called* Sam Altman's Number*, and*

$$2^{R_{\mathcal{V}}} < S_{\mathcal{V}}, \qquad R_{\mathcal{V}} < S_{\mathcal{V}}.$$

The name is a label; it supplies no extra bits to the oracle. The mathematical content lies in fixing the syntax, semantic input, machine encoding, and comparison witness well enough that another reader can reproduce the same definition.

Rayo explicitly uses the intended interpretation of set theory and warns that different models need not give the same value [1]. We retain that interpretation throughout. Our proof does not select a set-sized model and rename its value $R_{\mathcal{V}}$. It first reduces the semantic input to a set of natural-number data, then applies a theorem about *every* such input. Section 8 states exactly which parts are kernel-checked and which constitute the mathematical interpretation of those certificates.

The construction has no circular appeal to $S_{\mathcal{V}}$ or numerical input named "Rayo." Its order is: finite syntax, the intended truth profile, a base oracle, finite halting iterates, their join, and a finite output maximum.

2 Finite syntax with an infinite supply of variable names

The surface language has atoms $x_i = x_j$ and $x_i \in x_j$, together with $(\neg p)$, $(p \wedge q)$, and $\exists x_i(p)$. Each variable occurrence is one variable symbol; its subscript identifies the variable. This gives the token costs

$$\begin{aligned} \ell(x_i = x_j) = \ell(x_i \in x_j) &= 3, \\ \ell(\neg p) &= \ell(p) + 3, \\ \ell(p \wedge q) &= \ell(p) + \ell(q) + 3, \\ \ell(\exists x_i p) &= \ell(p) + 4. \end{aligned}$$

The costs include the parentheses displayed in the grammar. They use the five formula-building forms in Rayo's definition. We rename his distinguished free variable x_1 to x_0; variable names do not change token counts.

An infinite alphabet of variable names makes a direct list of all surface strings inconvenient. We therefore resolve binding into coordinates. Let $\mathcal{F}_k$ be the finite-term grammar

$$p ::= (i = j) \mid (i \in j) \mid \neg p \mid (p \wedge q) \mid \exists p, \qquad i, j < k,$$

where the child of an existential lies in $\mathcal{F}_{k+1}$. The new bound variable has coordinate zero; the preceding coordinates shift by one. The same token costs apply, and the sole free coordinate of $\mathcal{F}_1$ is zero.

For a membership structure and an assignment s, satisfaction is defined by the expected atomic clauses, negation and conjunction, and

$$\mathrm{Sat}(\exists p, s) \quad \Longleftrightarrow \quad \exists x \ \mathrm{Sat}(p, x :: s).$$

Every quantifier in an intended interpretation ranges over all sets. This notation is used externally for each fixed formula; it does not postulate one first-order truth predicate inside $\mathcal{V}$. For each fixed formula, these clauses agree with the satisfaction clauses in Rayo's second-order presentation by structural induction on that formula.

Lemma 2.1 (Normalization and finite enumeration)**.** *Every surface formula with only x_0 free translates to a formula of $\mathcal{F}_1$ with the same token count and the same satisfaction condition. For each natural b, the collection*

$$D_b = \{p \in \mathcal{F}_1 : \ell(p) < b\}$$

is finite and has a specified effective list enumeration.

Proof. Maintain a map from variable names to available coordinates. Under $\exists x_i$, send i to zero and shift every other coordinate. This handles shadowing as well as fresh names. Structural induction proves preservation of size and satisfaction, since extending a coordinate assignment gives exactly the corresponding surface assignment update.

Define depth to be one for atoms, one plus the child depth for negation and existential quantification, and one plus the maximum child depth for conjunction. Induction gives $\mathrm{depth}(p) \leq \ell(p)$. Let $E(k, 0)$ be the empty list. In the order specified below, $E(k, d+1)$ is the concatenation of:

1. equality atoms $(i = j)$, with i increasing first as the outer loop and j increasing in its inner loop;
2. membership atoms $(i \in j)$ in the same order;
3. $\neg p$ for p in $E(k, d)$;

4. $p \wedge q$, with p in $E(k, d)$ as the outer loop and q in that list as the inner loop;
5. $\exists p$ for p in $E(k + 1, d)$.

List order is preserved at every step, and duplicates are retained. Induction on d proves that membership in $E(k, d)$ is equivalent to depth at most d. Filter $E(1, b)$ by $\ell(p) < b$ to obtain the ordered list L_b. Its underlying set is D_b. □

For $p \in \mathcal{F}_1$, let U_p be the literal formula

$$\mathsf{U}_p(x) := p(x) \wedge \forall y\,(p(y) \longrightarrow y = x). \tag{1}$$

Universal quantification and implication abbreviate $\neg\exists\neg$ and $\neg(p \wedge \neg q)$, respectively, with capture-avoiding substitution. Thus U_p uses only the original five forms. It is equivalent to $\forall y\,(p(y) \leftrightarrow y = x)$.

Lemma 2.2 (All-object uniqueness)**.** *For every p, the sequent $\mathsf{U}_p(x), \mathsf{U}_p(y) \vdash x = y$ has an explicit first-order natural-deduction derivation.*

Proof. The first premise gives $p(x)$. The second gives $\forall z\,(p(z) \to z = y)$. Instantiate at x and apply implication elimination. These steps are represented directly by the constructors of `Derivation`; `unique_same` certifies the resulting sequent. □

The quantifier over y in (1) is essential. A formula with exactly the two solutions 0 and ω has one natural solution, but it does not uniquely name an object and contributes no named natural here.

3 The intended profile is a set of natural-number data

Write $\bar{n}$ for the finite von Neumann ordinal representing n: $\bar{0} = \varnothing$ and $\overline{n+1} = \bar{n} \cup \{\bar{n}\}$. Induction shows that the elements of $\bar{n}$ are exactly the representatives $\bar{m}$ with $m < n$. Each successor contains its predecessor, and the membership relation on ordinals is irreflexive; hence these representatives are distinct and $\bar{m} \in \bar{n}$ exactly when $m < n$.

For any budget b, define the finite syntactic support

$$K_b = D_b \cup \{\mathsf{U}_p : p \in D_b\}.$$

The formulas U_p need not themselves fit the original budget. They belong to the semantic input used to compare with that budget. This is a finite extension of syntax, without a self-reference: indeed $\ell(\mathsf{U}_p) = 2\ell(p) + 25$.

Definition 3.1 (Intended profile)**.** *The profile $T_{\mathcal{V}} : \mathcal{F}_1 \times \mathbb{N} \to \{\text{false}, \text{true}\}$ is*

$$T_{\mathcal{V}}(q, n) \quad \Longleftrightarrow \quad q \in K_G \text{ and } \mathcal{V} \models q[\bar{n}].$$

It is false outside K_G.

This definition requires truth for finitely many fixed formula templates, each evaluated at all natural parameters. It is not a finite table of truth values: an ordinary formula row can be infinite. To see why the profile is a set without assuming a global truth predicate, list the distinct members of K_G as $q_0, \ldots, q_{t-1}$. For each fixed q_i, Separation forms

$$A_i = \{n \in \omega : q_i(n)\}.$$

The formula q_i is inserted literally into this Separation instance. The finite union $\bigcup_{i<t}\{i\} \times A_i$ codes the required profile. There is no quantifier over formula codes inside a uniform assertion of set truth. Full truth for all first-order set-theory formulas is unnecessary.

For arbitrary profiles $T : \mathcal{F}_1 \times \mathbb{N} \to \mathrm{Prop}$, define

$$\mathrm{Adm}_T(p, n) := T(\mathsf{U}_p, n) \ \wedge \ \forall m\,(T(\mathsf{U}_p, m) \to m = n). \tag{2}$$

Every row of Adm_T is single-valued, even when T is an incoherent input. This totalization makes the later arithmetic theorem unconditional.

For the intended profile, Lemma 2.2 and injectivity of finite ordinals show

$$p \in D_G \quad \Longrightarrow \quad \mathrm{Adm}_{T_{\mathcal{V}}}(p, n) \iff \mathcal{V} \models \mathsf{U}_p[\bar{n}]. \tag{3}$$

Thus totalization removes no intended name. Notice that uniqueness among natural parameters in (2) is only a safeguard for arbitrary input profiles: all-object uniqueness is already inside the formula U_p.

Let

$$V_T(b) = \{n : \exists p \in D_b \ \ \mathrm{Adm}_T(p, n)\}, \qquad B_T(b) = \begin{cases} 0, & V_T(b) = \varnothing, \\ 1 + \max V_T(b), & V_T(b) \neq \varnothing. \end{cases}$$

The set $V_T(b)$ is finite because D_b is finite and each admitted row has at most one value. Define the comparison predicate

$$\mathrm{Below}_T(b, n) := \forall p \in D_b \ \forall m\,(\mathrm{Adm}_T(p, m) \to m < n). \tag{4}$$

Lemma 3.2 (Least strict bound)**.** *For every profile T and natural b, n, $\mathrm{Below}_T(b, n)$ holds exactly when $B_T(b) \leq n$.*

Proof. For a nonempty finite set, every element is less than one plus its maximum, and any strict upper bound is at least that successor. For the empty set the predicate is vacuous and its least natural solution is zero. □

All definitions below depend only on T restricted to K_b. For clarity, put $T^{[b]}(q, n) := (q \in K_b \wedge T(q, n))$. The support lemmas in `ProfileSupport.lean` prove that restriction leaves the admitted values, their boundary, the base oracle, and the final number unchanged.

4 A completely specified program language

We fix the numerical pairing function used by Mathlib:

$$\langle a, b\rangle = \begin{cases} b^2 + a, & a < b, \\ a^2 + a + b, & a \geq b. \end{cases} \tag{5}$$

Its inverse is $(\pi_0(n), \pi_1(n))$. Explicitly, if $s = \lfloor \sqrt{n} \rfloor$ and $r = n - s^2$, the inverse is (r, s) for $r < s$, and $(s, r - s)$ otherwise. The pairing is a bijection $\mathbb{N}^2 \to \mathbb{N}$.

4.1 Ordinary partial-recursive programs

Ordinary programs c have constructors

$$Z, \ S, \ L, \ R, \ P(c, d), \ C(c, d), \ Q(c, d), \ M(c).$$

They are Mathlib's `Nat.Partrec.Code`. On input n, the first four return 0, $n+1$, $\pi_0(n)$, and $\pi_1(n)$. Pairing returns the pair of two defined outputs. Composition $C(c,d)$ first runs d and then c. Primitive recursion is

$$\begin{aligned}\mathrm{ev}(Q(c,d),\langle a,0\rangle) &= \mathrm{ev}(c,a),\\ \mathrm{ev}(Q(c,d),\langle a,k+1\rangle) &= \mathrm{ev}\big(d,\langle a,\langle k,\mathrm{ev}(Q(c,d),\langle a,k\rangle)\rangle\rangle\big).\end{aligned}$$

Every displayed evaluation propagates divergence from a required earlier evaluation. On input $\langle a,m\rangle$, $M(c)$ returns the least $n \geq m$ such that $\mathrm{ev}(c,\langle a,n\rangle) = 0$, provided each trial from m through n terminates. If an earlier trial diverges, a later zero is not reached.

The atomic ordinary codes are $e(Z) = 0$, $e(S) = 1$, $e(L) = 2$, and $e(R) = 3$. The compound codes are fixed by

$$\begin{aligned}e(P(c,d)) &= 4\langle e(c),e(d)\rangle + 4,\\ e(C(c,d)) &= 4\langle e(c),e(d)\rangle + 6,\\ e(Q(c,d)) &= 4\langle e(c),e(d)\rangle + 5,\\ e(M(c)) &= 4e(c) + 7.\end{aligned}$$

This encoding has a total inverse. After the four atomic codes, decode $n+4$ according to n mod 4: residues $0,1,2,3$ select pairing, primitive recursion, composition, and minimization. The quotient $\lfloor n/4\rfloor$ supplies either a paired pair of child codes or the single child code.

4.2 Oracle programs

For a total oracle $o : \mathbb{N} \to \mathbb{N}$, our programs are

$$p ::= \mathsf{prim}(c) \mid \mathsf{query} \mid \mathsf{guard} \mid \mathsf{pair}(p,q) \mid \mathsf{comp}(p,q) \mid \mathsf{prec}(p,q) \mid \mathsf{find}(p).$$

The interpreter $\mathrm{ev}_o(p,n)$ is a deterministic partial natural number. $\mathsf{prim}(c)$ runs the ordinary program, and query returns $o(n)$. The guard returns zero at input zero and diverges otherwise. Pairing, composition, and primitive recursion follow the preceding equations with oracle evaluations. Finally,

$$\mathrm{ev}_o(\mathsf{find}(p),a) = \mu n\,[\mathrm{ev}_o(p,\langle a,n\rangle) = 0],$$

where all earlier trials must terminate with nonzero outputs. These clauses are exactly the partial semantics in `OracleMachine.eval`.

The oracle encoding is

$$\begin{aligned}\mathrm{enc}(\mathsf{prim}(c)) &= 8e(c), & \mathrm{enc}(\mathsf{query}) &= 1, & \mathrm{enc}(\mathsf{guard}) &= 2,\\ \mathrm{enc}(\mathsf{pair}(p,q)) &= 8\langle \mathrm{enc}(p),\mathrm{enc}(q)\rangle + 3,\\ \mathrm{enc}(\mathsf{comp}(p,q)) &= 8\langle \mathrm{enc}(p),\mathrm{enc}(q)\rangle + 4,\\ \mathrm{enc}(\mathsf{prec}(p,q)) &= 8\langle \mathrm{enc}(p),\mathrm{enc}(q)\rangle + 5,\\ \mathrm{enc}(\mathsf{find}(p)) &= 8\,\mathrm{enc}(p) + 6.\end{aligned}$$

Decode by the remainder modulo eight and recursively decode the quotient or its two unpaired components. Tags one and two require quotient zero; tag seven is invalid. Tag zero uses the total ordinary decoder. An invalid child makes the entire code invalid. The code zero decodes as $\mathsf{prim}(Z)$. Recursive child codes are smaller, so this procedure terminates.

The checked round-trip theorem is $\mathrm{decode}(\mathrm{enc}(p)) = p$; hence the encoding is injective. Let $\mathrm{run}_o(e,n)$ run the decoded program, or diverge if the code is invalid. No undefined evaluator is assigned an artificial executable output.

5 The oracle hierarchy and the named number

Write $L_b = (p_0, \dots, p_{r-1})$ for the ordered list of Lemma 2.1. Define the base oracle by

$$O_{T,b}(\langle 0, n\rangle) = \begin{cases} 0, & \mathrm{Below}_T(b, n), \\ 1, & \text{otherwise}, \end{cases} \tag{6}$$

$$O_{T,b}(\langle i+1, n\rangle) = \begin{cases} 0, & i < r \text{ and } T(p_i, n), \\ 1, & \text{otherwise}. \end{cases} \tag{7}$$

Channel zero tests the comparison predicate. The other channels expose the ordinary description-truth rows, including duplicate channels when the list contains duplicates. The rows of U_p affect the comparison predicate; they are not separate direct query channels unless they themselves occur in L_b. This explains exactly why the support is K_b.

For every total oracle o, define its diagonal halting function

$$H(o)(e) = \begin{cases} 1, & \mathrm{run}_o(e, e)\downarrow, \\ 0, & \mathrm{run}_o(e, e)\uparrow. \end{cases}$$

The halting bit convention differs from the successful truth-query convention: halting has bit one, while a true base query has bit zero. Starting with o, form

$$J_0(o) = o, \qquad J_{k+1}(o) = H(J_k(o)), \qquad J_\omega(o)(\langle k, n\rangle) = J_k(o)(n).$$

The last expression is the tagged join of every finite stage, including stage zero. These are ordinary set-valued recursions once the base oracle is a set of natural-number data.

Proposition 5.1 (The join exceeds every finite stage in computational power)**.** *No program using oracle $J_k(o)$ computes $J_{k+1}(o)$ on every input. Every finite stage is computable using $J_\omega(o)$, and no finite stage computes the whole join, in the program language of Section 4.*

Proof. If a program p computed $H(o)$, form $d = \mathsf{comp}(\mathsf{guard}, p)$. At its own code, d would halt exactly when it did not halt. This proves the first assertion at every stage. A query after the address transformation $n \mapsto \langle k, n\rangle$ retrieves stage k from the join. If stage k computed the whole join, that same transformation would compute its successor. This contradicts the first assertion. □

Define the auxiliary total output

$$a_o(e) = \begin{cases} v, & \mathrm{run}_o(e, 0) = v, \\ 0, & \mathrm{run}_o(e, 0)\uparrow. \end{cases}$$

The zero fallback is used only when forming a finite maximum. For a bit budget $w \in \mathbb{N}$, put

$$\mathrm{BB}^+_o(w) = 1 + \max_{0 \le e < 2^w} a_o(e). \tag{8}$$

This bound measures numerical output in our encoding. The classical running-time Busy Beaver convention defines a different function. The numerical codes are strictly below 2^w, so leading-zero padding is permitted and the zero-bit convention admits only code zero.

Definition 5.2 (The construction)**.** *For every profile T, define*

$$S(T) = \mathrm{BB}^+_{J_\omega(O_{T^{[G]},G})}(G).$$

Sam Altman's Number is

$$\boxed{S_{\mathcal{V}} = S(T_{\mathcal{V}}).}$$

This is a finite natural: there are finitely many codes below 2^G, every halting code has a unique natural output, and a finite set of naturals has a maximum. The definition includes a hierarchy inaccessible to each of its finite stages and a maximum over the entire permitted program space. Its comparison proof will need only one program from that space.

The cutoff channel alone is computable with its finite threshold hardwired; we make no claim of noncomputability for that channel. Ordinary truth rows can carry additional information. The diagonal theorem about the hierarchy holds for every base, regardless of the complexity of those rows.

6 The actual comparison program

Let $z = \mathsf{prim}(Z)$ and $r = \mathsf{prim}(R)$. Define the address program and search program

$$A = \mathsf{pair}\big(z, \mathsf{pair}(z, r)\big), \qquad W = \mathsf{find}\big(\mathsf{comp}(\mathsf{query}, A)\big). \tag{9}$$

On a search trial input $\langle 0, n\rangle$, A returns $\langle 0, \langle 0, n\rangle\rangle$. The outer tag selects stage zero of the join; the inner tag selects the comparison channel of the base. Consequently

$$J_\omega(O_{T,b})(\langle 0, \langle 0, n\rangle\rangle) = 0 \quad \Longleftrightarrow \quad \mathrm{Below}_T(b, n).$$

Lemma 6.1 (Exact search output)**.** *For every profile T and every natural budget b,*

$$\mathrm{ev}_{J_\omega(O_{T,b})}(W, 0) = B_T(b).$$

Proof. Every address transformation and oracle query terminates. By Lemma 3.2, the tested predicate first holds at $B_T(b)$. Every earlier trial returns one and that trial returns zero, so the minimization semantics returns exactly $B_T(b)$. □

The search expression uses only the existing program constructors, and its behavior follows from their partial evaluation clauses. Its code is fixed independently of T, b, or any numerical Rayo value. The encoding equations give

$$\begin{aligned} \mathrm{enc}(z) &= 0, & \mathrm{enc}(r) &= 24, \\ \mathrm{enc}(\mathsf{pair}(z, r)) &= 4611, & \mathrm{enc}(A) &= 170090571, \\ \mathrm{enc}(W) &= 1851571349958786726 < 2305843009213693952 = 2^{61}. \end{aligned}$$

The final equality and inequality are kernel-checked using ordinary decision procedures, without native evaluation as a proof axiom.

We next specify an ordinary program that exponentiates the search output. Let $I = P(L, R)$, $c_0 = Z$, and $c_{k+1} = C(S, c_k)$, so that I is the identity program and c_k is the constant-k program. Define

$$\begin{aligned} \mathsf{Add} &= Q\big(I, C(S, C(R, R))\big), \\ \mathsf{Double} &= C\big(\mathsf{Add}, P(I, I)\big), \\ \mathsf{PowRec} &= Q\big(c_1, C(\mathsf{Double}, C(R, R))\big), \\ \mathsf{Pow2} &= C\big(\mathsf{PowRec}, P(c_0, I)\big), \\ W_{\exp} &= \mathsf{comp}\big(\mathsf{prim}(\mathsf{Pow2}), W\big). \end{aligned}$$

These are abbreviations for finite terms in the already specified grammars.

Lemma 6.2 (Exponential witness)**.** *For every profile T and budget b,*

$$\mathrm{ev}_{J_\omega(O_{T,b})}(W_{\exp}, 0) = 2^{B_T(b)}, \qquad \mathrm{enc}(W_{\exp}) < 2^{2048}.$$

Proof. Induction in the recursion equations gives $\mathrm{ev}(\mathsf{Add}, \langle a, n\rangle) = a + n$. Duplication followed by addition therefore gives $\mathrm{ev}(\mathsf{Double}, n) = 2n$. A second induction gives $\mathrm{ev}(\mathsf{PowRec}, \langle a, n\rangle) = 2^n$: the base value is one, and the step doubles the preceding value. Thus $\mathsf{Pow2}$ returns 2^n on input n, and composition with Lemma 6.1 proves the output claim. The code inequality follows by evaluating the encoding equations for these fixed terms; `GrowthCertificate.witness_code_lt` checks it in Lean. □

Theorem 6.3 (Unconditional arithmetic certificate)**.** *For every profile T, the number $S(T)$ is finite and*

$$B_{T^{[G]}}(G) < S(T), \qquad 2^{B_{T^{[G]}}(G)} < S(T).$$

No coherence, satisfaction, or set-theoretic interpretation hypothesis is needed.

Proof. The admission rule (2) makes every admitted row single-valued, so $B_{T^{[G]}}(G)$ exists. By Lemma 6.1, W outputs this boundary using the oracle in Definition 5.2. Since $61 \le G$, its code satisfies

$$\mathrm{enc}(W) < 2^{61} \le 2^G.$$

It is therefore among the programs maximized over in (8); the successor of that maximum is strictly larger than its output. Finiteness was established directly after Definition 5.2. For the stronger inequality use W_{exp} instead. Lemma 6.2 gives its output, and $2048 \le G$ gives $\mathrm{enc}(W_{\mathrm{exp}}) < 2^{2048} \le 2^G$. The same strict maximum argument applies. □

7 Applying the certificate to the original benchmark

Proof of Theorem 1.1. Fix the intended interpretation used to define $R_{\mathcal{V}}$. Suppose a surface formula $p(x_0)$ with $\ell(p) < G$ uniquely names the finite ordinal $\bar{n}$. Normalize it using Lemma 2.1, obtaining $q \in D_G$ with the same size and satisfaction condition. Thus q is true of $\bar{n}$ and of no other set, so $\mathcal{V} \models \mathsf{U}_q[\bar{n}]$.

The formula U_q belongs to K_G, hence $T_{\mathcal{V}}(\mathsf{U}_q, n)$ holds. Lemma 2.2 and finite-ordinal injectivity imply that no other natural parameter makes this row true. Therefore $\mathrm{Adm}_{T_{\mathcal{V}}}(q, n)$ holds and $n \in V_{T_{\mathcal{V}}}(G)$. In particular $n < B_{T_{\mathcal{V}}}(G)$.

Every original Rayo denotation is therefore below $B_{T_{\mathcal{V}}}(G)$. The normalization also proves that the original naming collection is finite, so its least strict bound exists. By the defining minimality of $R_{\mathcal{V}}$,

$$R_{\mathcal{V}} \le B_{T_{\mathcal{V}}}(G).$$

The profile $T_{\mathcal{V}}$ is already false outside K_G, so restricting it has no effect. Apply Theorem 6.3 to this set of natural-number data:

$$2^{R_{\mathcal{V}}} \le 2^{B_{T_{\mathcal{V}}}(G)} < S(T_{\mathcal{V}}) = S_{\mathcal{V}}.$$

The first inequality uses monotonicity of exponentiation. Since $R_{\mathcal{V}} \le 2^{R_{\mathcal{V}}}$, the asserted strict comparison with $R_{\mathcal{V}}$ follows as well. Finiteness of $S_{\mathcal{V}}$ follows from the same theorem. □

Only coverage is used: the proof does not assert equality between the surface and scoped naming collections. It also does not pass the class of all sets as a carrier to a Lean semantic theorem. The intended profile is obtained directly from the finite family of literal formulas, then supplied to the unconditional theorem about profiles on ordinary syntax and naturals.

The surface-language comparison itself also has a uniform profile formulation. Let $\mathrm{sc}(p)$ be the scope translation of a surface formula, with its free variable assigned coordinate zero, and put

$$N_T(b) = \{n : \exists p\ (\ell(p) < b\ \wedge\ \mathrm{FV}(p) = \{x_0\}\ \wedge\ \mathrm{Adm}_T(\mathrm{sc}(p), n))\}.$$

Define $R(T, b)$ to be zero when $N_T(b)$ is empty, and $1 + \max N_T(b)$ otherwise. This set is finite because $N_T(b) \subseteq V_T(b)$ by token preservation. Consequently $R(T, b)$ is its least strict natural bound and $R(T, b) \leq B_T(b)$. The same admission guard (2) handles arbitrary profiles: a row with several true natural parameters admits none of them.

For $T_{\mathcal{V}}$ and $b = G$, a surface formula contributes n to $N_{T_{\mathcal{V}}}(G)$ if and only if it uniquely names $\bar{n}$ under Rayo's original condition. Indeed, scope translation preserves satisfaction in both directions; equation (3) then identifies admission with all-object unique naming. Thus

$$R(T_{\mathcal{V}}, G) = R_{\mathcal{V}}. \tag{10}$$

This identifies the original surface benchmark with its profile encoding, without asserting that every scoped formula is an eligible surface name. The module `OriginalBenchmark.lean` checks the least-bound property and the direct uniform comparison

$$\forall T, \quad 2^{R(T,G)} < S(T).$$

Its proof combines $R(T, G) \leq B_T(G)$ with Theorem 6.3 and support invariance. Equation (10) supplies the intended interpretation of this additional certificate; the definition of $S_{\mathcal{V}}$ is unchanged.

The displayed formula templates can be astronomically large. Their generation is an ordinary finite recursive definition; the proof uses induction on that definition and never expands the googol-sized enumeration in memory. The finite bound on formula templates is also why no uniform truth predicate or unbounded set-theoretic satisfaction class is needed.

8 What the formalization certifies

The boundary comparison declarations are `FiniteProfile.fixed_comparison` and `FiniteProfile.samAltman_comparison`. They quantify over every profile. The latter states, in the notation of this paper,

$$\forall T, \quad B_{T^{[G]}}(G) < S(T).$$

The stronger declaration `GrowthCertificate.exponential_boundary_lt_samAltman` states

$$\forall T, \quad 2^{B_{T^{[G]}}(G)} < S(T).$$

The surface benchmark is covered directly by `OriginalBenchmark.exponential_originalRayo_lt_samAltman`, whose statement is $\forall T,\ 2^{R(T,G)} < S(T)$. Its input is a profile on ordinary syntax and naturals, with no set-carrier parameter or semantic hypothesis. The intended profile $T_{\mathcal{V}}$ is specified mathematically in Definition 3.1; Lean neither chooses that interpretation nor computes its truth values. The proof in the preceding section applies the checked theorem to that specified set. This external interpretation is part of the intended-semantics statement, and is not a closed Lean declaration identifying an internal constant with the original universe's Rayo value.

The logical bridge has its own certificate. `FOLDerivation.lean` defines an inductive natural-deduction relation, proves its structural soundness, and constructs the uniqueness derivation of Lemma 2.2. Its rules include hypotheses, weakening, renaming, the usual connective and quantifier rules, and equality substitution. No rule imports an arbitrary Lean semantic proof as an object-language derivation. For each fixed formula, the finite logical derivation is valid under the intended interpretation by those same rules; no domain-size condition occurs in the uniqueness argument.

Claim	Principal certificate or interpretation step
Finite description coverage	`Formula.mem_enumerate`, `Formula.mem_enumerateList`
Raw binding and symbol preservation	`RawFormula.sat_scope`, `RawFormula.symbols_scope`
Original assignment condition	`RawFormula.denotes_iff_winning_condition`
All-object unique naming	`Formula.sat_unique`; syntactic `Derivation.unique_same`
Finite support suffices	`FiniteProfile.base_congr`, `FiniteProfile.restrict_number`
Program encoding and partial search	`decode_encode`, `mem_eval_find`, `eval_searchWith_of_least`
Exact witness and code bound	`FiniteProfile.witness_output`, `Candidate.witness_code_lt`
Strict hierarchy	`not_computes_successor`, `not_computes_join`
Uniform final comparison	`FiniteProfile.samAltman_comparison`
Exponential amplification	`GrowthCertificate.lean`: `exponential_boundary_lt_samAltman`
Exact surface benchmark	`OriginalBenchmark.lean`: `originalRayo_isLeast`
Direct surface comparison	`OriginalBenchmark.lean`: `exponential_originalRayo_lt_samAltman`
Intended input exists	Finitely many Separation instances and a finite union, as specified in Section 3
Original intended comparison	The application proved in Section 7, using the preceding syntactic and arithmetic certificates

There are additional model-based validation theorems. The compositional profile in an arbitrary type-carrier structure satisfies the same interface; `OriginalBenchmark.originalRayo_semanticProfile` identifies $R(T, b)$ with the independently defined `Candidate.rawRayo` there, assuming an injective representation of naturals. Its `semantic_exponential_comparison` checks the stronger comparison in that interpretation. A separate `ZFSet` instance checks an explicit first-order natural-number formula and its finite-ordinal interpretation. These are checks of the definitions, with their model qualification retained. No identification of `ZFSet` with $\mathcal{V}$ is used in the main proof.

The project permits only the ordinary Lean foundational dependencies `propext`, `Classical.choice`, and `Quot.sound` in the audited declarations. The uniqueness certificate uses only the first and third. The accepted source contains no proof placeholders, custom mathematical axioms, or `native_decide`. Kernel checking establishes the stated formal theorems; the preceding interpretation argument explains their exact relation to the intended-semantics number.

9 Relation to earlier constructions

Relativized Busy Beaver bounds and their iteration are established ideas. Aaronson describes hierarchies obtained by giving programs halting oracles for preceding levels [2]. Our output measure and integer encoding are specified independently, so no equality with a classical Turing-machine Busy Beaver value is asserted.

The use of stronger semantic resources has substantial background in the study of higher-order languages; Linnebo and Rayo discuss semantic and ontological hierarchies in that setting [3]. We need only a fixed finite family of set-theoretic formula templates. The philosophical choice of the intended interpretation remains the one made in Rayo's definition.

Fish's account of Fish number 7 explicitly combines extensions of Rayo's language with oracle and hierarchy ideas to exceed the Rayo benchmark [4]. This is direct prior overlap in method. We make no ordering claim between $S_{\mathcal{V}}$ and Fish number 7, and no claim that naming a number above Rayo is new. The contribution here is the particular fully specified construction and its accompanying checked certificates. The result settles the comparison posed in this paper; it does

not impose a ceiling on future definitions of finite numbers.

A Pinned sources and reproduction

The source package is in `research/sam-altman-number/` of https://github.com/NoahWLono/Sam-Altmans-Number. Its dependency pins are Lean `leanprover/lean4:v4.35.0` and Mathlib revision

`e91b128082cc221d6432a6a5aadcd3243af8e76c`.

The repository's `lean-toolchain` and `lake-manifest.json` fix the toolchain and dependency revisions. The package's recorded hashes identify the reviewed sources and exported manuscript.

The definitions in this paper follow these pinned implementations:

- `Mathlib/Data/Nat/Pairing.lean`: `Nat.pair` and `Nat.unpair`.
- `Mathlib/Computability/PartrecCode.lean`: `Nat.Partrec.Code`, `encodeCode`, `ofNatCode`, and `eval`.
- `FormulaSemantics.lean` and `BoundedTruth.lean`: surface/scoped syntax, token costs, normalization, list order, and literal unique-name formulas.
- `OracleMachine.lean`, `OracleSearch.lean`, and `OracleHierarchy.lean`: oracle code, partial semantics, search, finite halting iterates, and the join.
- `Candidate.lean`: the fixed address/search witness and its exact code.
- `GrowthCertificate.lean`: ordinary addition and exponentiation, the exponential witness, its code bound, and the stronger comparison.
- `FiniteProfile.lean` and `ProfileSupport.lean`: admission, the base oracle, support restriction, and the named number.
- `OriginalBenchmark.lean`: the exact surface-profile naming collection, its least strict bound, and the direct exponential comparison.
- `FOLDerivation.lean`: explicit syntactic certificates and soundness.

From the repository root, after restoring the pinned dependencies, run the verification script to rebuild all thirteen modules, including `GrowthCertificate.lean` and `OriginalBenchmark.lean`, in dependency order and audit the resulting axiom reports. This checks the proofs without enumerating D_G, computing the intended profile, or evaluating $S_{\mathcal{V}}$.

```
python3 research/sam-altman-number/verify.py
```

The source files print their principal transitive axiom dependencies using `#printaxioms`. Check the complete build output for errors, warnings, proof-placeholder dependencies, or any dependency outside the stated allowed set. The package verification script and its recorded evidence provide the corresponding source and dependency audit. The LaTeX source is `sam-altmans-number.tex`; compiling it produces the matching paper.

References

[1] Agustín Rayo. *Big Number Duel*. https://web.mit.edu/arayo/www/bignums.html. Final displayed winning-entry definition and interpretation note. Accessed 15 October 2026.

[2] Scott Aaronson. *The Busy Beaver Frontier*, especially Section 3. https://scottaaronson.com/bb.pdf. Accessed 15 October 2026.

[3] Øystein Linnebo and Agustín Rayo. *Hierarchies Ontological and Ideological*. https://web.mit.edu/arayo/www/tt-g.pdf. Accessed 15 October 2026.

[4] Fish. *Googology in Japan—exploring large numbers*, sections on Fish numbers 4 and 7. https://gyafun.jp/ln/en.html. Accessed 15 October 2026.